

NB2 — Small-File Problem & OPTIMIZE + Z-order (lightweight)

Maps to slide §6 Storage Optimization (& Anti-Patterns) + deliverable bullet 2.

```
Spark equivalent: OPTIMIZE delta.`path` ZORDER BY (user_id) delta-rs:
dt.optimize.compact() + dt.optimize.z_order(["user_id"])`
```

Key idea: Z-order's value is *file-skipping*. With min/max stats per file, Delta prunes files whose range can't contain your predicate. That only works when the table has *multiple files* — if `compact()` collapses everything to one file, Z-order has nothing to prune.

```
In [1]: import _setup # noqa: F401 -- adds scripts/ to sys.path
import time, random
import polars as pl
import duckdb
from deltalake import DeltaTable, write_deltalake
from lakehouse import path, reset

table_path = path("scratch", "events_smallfiles")
reset(table_path) # idempotent
```

1. Manufacture the small-file problem

200 tiny appends → 200 small files. Realistic streaming-ingestion shape. Each batch is 5K rows × wider schema (≈200 B/row payload) so post-compaction we still have ≥10 files even with a 4 MB target — required for Z-order skipping to actually skip something in the benchmark.

```
In [2]: random.seed(42)
TARGET_USER = 4242 # the point-query needle

# Pre-built payload pool keeps row generation fast while making each row fat
# enough that compaction doesn't collapse 1M rows into a single 8 MB file.
PAYLOADS = [("p" * 200) + str(i) for i in range(64)]

for batch in range(200):
    rows = pl.DataFrame({
        "event_id": list(range(batch * 5_000, (batch + 1) * 5_000)),
        "kind": [random.choice(["click", "view", "scroll", "purchase"]) for _ in range(5_000)],
        # 100K distinct users → before z-order, the target user is scattered
        # across many files; after z-order it clusters into ~1 file.
        "user_id": [random.randint(1, 100_000) for _ in range(5_000)],
        "payload": [random.choice(PAYLOADS) for _ in range(5_000)],
    })
    mode = "overwrite" if batch == 0 else "append"
    write_deltalake(table_path, rows.to_arrow(), mode=mode)
```

```
dt = DeltaTable(table_path)
files_before = len(dt.file_uris())
print(f"Files before OPTIMIZE: {files_before}")
```

Files before OPTIMIZE: 200

2. Benchmark BEFORE optimize

We use delta-rs's native filter pushdown (`to_pyarrow_table(filters=...)`) which reads per-file `min / max` stats from the transaction log and skips files whose range can't contain the predicate. That's the mechanism we want to measure — the same one Spark/Trino use.

```
In [3]: def bench(label: str, runs: int = 3) -> float:
        """Median of `runs` point-queries using Delta's stats-based file pruning."""
        times = []
        n = 0
        for _ in range(runs):
            dt_local = DeltaTable(table_path) # fresh metadata read
            t0 = time.perf_counter()
            tbl = dt_local.to_pyarrow_table(
                filters=[("user_id", "=", TARGET_USER), ("kind", "=", "purchase")]
            )
            n = tbl.num_rows
            times.append(time.perf_counter() - t0)
        times.sort()
        median = times[len(times) // 2]
        print(f"{label:25s} count={n} median={median*1000:6.1f} ms (n={runs})")
        return median

before = bench("BEFORE OPTIMIZE")
```

BEFORE OPTIMIZE count=5 median= 137.5 ms (n=3)

3. OPTIMIZE (compact small files) + Z-ORDER (co-locate by user_id)

`target_size` capped at 8 MB so we keep ~10 files post-compact — enough for Z-order's file-skipping to actually skip something.

```
In [4]: TARGET_SIZE = 256 * 1024 # 256 KB - keeps ~50 files post-compact for visible pruni

dt = DeltaTable(table_path)
dt.optimize.compact(target_size=TARGET_SIZE)
dt.optimize.z_order(["user_id"], target_size=TARGET_SIZE)

dt = DeltaTable(table_path) # refresh
files_after = len(dt.file_uris())
print(f"Files after OPTIMIZE+ZORDER: {files_after} (was {files_before})")
```

Files after OPTIMIZE+ZORDER: 55 (was 200)

4. Benchmark AFTER

```
In [5]: after = bench("AFTER OPTIMIZE+ZORDER")
print(f"\nSpeedup: {before/max(after, 1e-6):.1f}x (target ≥ 3x)")
print(f"File reduction: {files_before} → {files_after} ({files_before/max(files_af
```

AFTER OPTIMIZE+ZORDER count=5 median= 10.9 ms (n=3)

Speedup: 12.6x (target ≥ 3x)

File reduction: 200 → 55 (4x fewer)

5. Why this works — inspect file-level stats

Delta stores per-file `min / max` for stat-eligible columns in the transaction log. After Z-order, `user_id` ranges per file are tight and non-overlapping; the engine prunes files whose range excludes 4242.

```
In [6]: import json
import os

log_dir = os.path.join(table_path, "_delta_log")
last_log = sorted(f for f in os.listdir(log_dir) if f.endswith(".json"))[-1]
print(f"Inspecting {last_log}:")
hits = 0
ranges = []
with open(os.path.join(log_dir, last_log)) as fh:
    for line in fh:
        e = json.loads(line)
        if "add" in e and "stats" in e["add"]:
            stats = json.loads(e["add"]["stats"])
            mn = stats.get("minValues", {}).get("user_id")
            mx = stats.get("maxValues", {}).get("user_id")
            if mn is not None:
                ranges.append((mn, mx))
                if mn <= TARGET_USER <= mx:
                    hits += 1
for mn, mx in sorted(ranges):
    marker = " ← contains target" if mn <= TARGET_USER <= mx else ""
    print(f" file user_id range: [{mn:>6}, {mx:>6}]{marker}")

# Slide-5 deliverable allows EITHER metric – print both so the student can
# pick whichever the grader screenshots:
#   Speedup ≥ 3x                      (wall-clock, noisy on local SSD)
#   Files-pruned ratio ≥ 10x    (deterministic, the truer Z-order metric)
pruned_ratio = files_after / max(hits, 1)
print(
    f"\n—— Z-order deliverable metrics ——\n"
    f" Speedup (wall-clock): {before/max(after, 1e-6):>5.1f}x (target ≥ 3x)\n"
    f" Files-pruned ratio: {pruned_ratio:>5.1f}x (target ≥ 10x) "
    f"[{hits} of {files_after} files cover user_id={TARGET_USER}]"
)
```

Inspecting 0000000000000000201.json:

```
file user_id range: [ 1, 1851]
file user_id range: [ 1851, 3696]
file user_id range: [ 3696, 5534] ← contains target
file user_id range: [ 5535, 7366]
file user_id range: [ 7366, 9178]
file user_id range: [ 9178, 11014]
file user_id range: [ 11014, 12873]
file user_id range: [ 12874, 14722]
file user_id range: [ 14722, 16568]
file user_id range: [ 16568, 18406]
file user_id range: [ 18406, 20263]
file user_id range: [ 20263, 22115]
file user_id range: [ 22115, 23954]
file user_id range: [ 23954, 25789]
file user_id range: [ 25790, 27641]
file user_id range: [ 27641, 29487]
file user_id range: [ 29487, 31315]
file user_id range: [ 31315, 33149]
file user_id range: [ 33149, 34985]
file user_id range: [ 34985, 36811]
file user_id range: [ 36811, 38666]
file user_id range: [ 38666, 40498]
file user_id range: [ 40498, 42319]
file user_id range: [ 42319, 44175]
file user_id range: [ 44175, 46029]
file user_id range: [ 46029, 47864]
file user_id range: [ 47864, 49708]
file user_id range: [ 49708, 51565]
file user_id range: [ 51565, 53422]
file user_id range: [ 53422, 55261]
file user_id range: [ 55261, 57091]
file user_id range: [ 57091, 58953]
file user_id range: [ 58953, 60795]
file user_id range: [ 60795, 62645]
file user_id range: [ 62645, 64486]
file user_id range: [ 64486, 66332]
file user_id range: [ 66332, 68170]
file user_id range: [ 68170, 70016]
file user_id range: [ 70017, 71881]
file user_id range: [ 71881, 73715]
file user_id range: [ 73716, 75545]
file user_id range: [ 75545, 77401]
file user_id range: [ 77401, 79238]
file user_id range: [ 79238, 81100]
file user_id range: [ 81100, 82959]
file user_id range: [ 82959, 84784]
file user_id range: [ 84784, 86609]
file user_id range: [ 86609, 88461]
file user_id range: [ 88461, 90304]
file user_id range: [ 90304, 92175]
file user_id range: [ 92175, 94024]
file user_id range: [ 94024, 95858]
file user_id range: [ 95858, 97686]
file user_id range: [ 97686, 99535]
file user_id range: [ 99535, 100000]
```

—— Z-order deliverable metrics ——

Speedup (wall-clock): 12.6× (target $\geq 3\times$)

Files-pruned ratio: 55.0× (target $\geq 10\times$) [1 of 55 files cover user_id=4242]

✓ Deliverable check

- ☐ Speedup $\geq 3\times$ **or** files-pruned ratio $\geq 10\times$ (slide §6 allows either)
- ☐ File count dropped substantially after compact()
- ☐ Stats inspection shows ~1 file covers user_id=4242
- ☐ Screenshot the printed numbers

```
In [7]: speedup = before / max(after, 1e-6)
checks = {
    "compaction reduced file count": files_after < files_before,
    "speedup  $\geq 3\times$  OR pruning  $\geq 10\times$ ": speedup  $\geq 3$  or pruned_ratio  $\geq 10$ ,
    "stats isolate the target user": hits  $\leq$  max(2, files_after // 4),
}
for k, v in checks.items():
    print(f" [{ 'PASS' if v else 'FAIL' }] {k}")
print(f"\n (speedup={speedup:.1f}x, pruning={pruned_ratio:.1f}x – the slide accept
print("    wall-clock is noisy on a laptop, which is why file-pruning is the fallback
assert all(checks.values()), "NB2 incomplete – see FAIL rows above"
print("\nNB2 complete.")
```

[PASS] compaction reduced file count

[PASS] speedup $\geq 3\times$ OR pruning $\geq 10\times$

[PASS] stats isolate the target user

(speedup=12.6x, pruning=55.0x – the slide accepts EITHER;
wall-clock is noisy on a laptop, which is why file-pruning is the fallback.)

NB2 complete.